

Migrations allow for you to define sets of schema changes so upgrading a database is a breeze.

Migration CLI

The migration CLI is bundled with the knex install, and is driven by the [node-liftoff](#) module. To install globally, run:

```
$ npm install knex -g
```

bash

The migration CLI accepts the following general command-line options. You can view help text and additional options for each command using `--help`. E.g. `knex migrate:latest --help`.

- `--debug` : Run with debugging
- `--knexfile [path]` : Specify the knexfile path
- `--knexpath [path]` : Specify the path to the knex instance
- `--cwd [path]` : Specify the working directory
- `--client [name]` : Set the DB client
- `--connection [address]` : Set the DB connection
- `--migrations-table-name` : Set the migration table name
- `--migrations-directory` : Set the migrations directory
- `--env` : environment, default: `process.env.NODE_ENV || development`
- `--esm` : [Enables ESM module interoperability](#)
- `--help` : Display help text for a particular command and exit.

Migrations use a **knexfile**, which specify various configuration settings for the module. To create a new knexfile, run the following:

```
$ knex init  
  
# or for .ts  
  
$ knex init -x ts
```

bash

will create a sample knexfile.js – the file which contains our various database configurations. Once you have a knexfile.js, you can use the migration tool to create migration files to the specified directory (default migrations). Creating new migration files can be achieved by running:

```
$ knex migrate:make migration_name  
  
# or for .ts  
  
$ knex migrate:make migration_name -x ts
```

bash

- you can also create your migration using a specific stub file, this serves as a migration template to speed up development for common migration operations
- if the --stub option is not passed, the CLI will use either the knex default stub for the chosen extension, or the config.stub file

```
$ knex migrate:make --stub  
  
# or  
  
$ knex migrate:make --stub
```

bash

- if a stub path is provided, it must be relative to the knexfile.[js, ts, etc] location
- if a file name is used, the stub is selected by its file name. The CLI will look for this file in the config.migrations.directory folder. If the config.migrations.directory is not defined, this operation will fail

Once you have finished writing the migrations, you can update the database matching your `NODE_ENV` by running:

```
$ knex migrate:latest
```

bash

You can also pass the `--env` flag or set `NODE_ENV` to select an alternative environment:

```
$ knex migrate:latest --env production
```

or

```
$ NODE_ENV=production knex migrate:latest
```

bash

To rollback the last batch of migrations:

```
$ knex migrate:rollback
```

bash

To rollback all the completed migrations:

```
$ knex migrate:rollback --all
```

bash

To run the next migration that has not yet been run

```
$ knex migrate:up
```

bash

To run the specified migration that has not yet been run

```
$ knex migrate:up 001_migration_name.js
```

bash

To undo the last migration that was run

```
$ knex migrate:down
```

bash

To undo the specified migration that was run

```
$ knex migrate:down 001_migration_name.js
```

bash

To list both completed and pending migrations:

```
$ knex migrate:list
```

bash

Seed files

Seed files allow you to populate your database with test or seed data independent of your migration files.

Seed CLI

To create a seed file, run:

```
$ knex seed:make seed_name
```

bash

Seed files are created in the directory specified in your knexfile.js for the current environment. A sample seed configuration looks like:

```
module.exports = {  
  // ...  
  development: {  
    client: {  
      /* ... */  
    },  
    connection: {  
      /* ... */  
    },  
    seeds: {  
      directory: './seeds/dev',  
    },  
  },  
  // ...  
};
```

js

If no `seeds.directory` is defined, files are created in `./seeds`. Note that the seed directory needs to be a relative path. Absolute paths are not supported (nor is it good practice).

To run seed files, execute:

```
$ knex seed:run
```

bash

Seed files are executed in alphabetical order. Unlike migrations, *every* seed file will be executed when you run the command. You should design your seed files to reset tables as needed before inserting data.

To run specific seed files, execute:

```
$ knex seed:run --specific=seed-filename.js --specific=another-seed-filename.js
```

bash

knexfile.js

A `knexfile.js` generally contains all of the configuration for your database. It can optionally provide different configuration for different environments. You may pass a `--knexfile` option to any of the command line statements to specify an alternate path to your `knexfile`.

Basic configuration

```
module.exports = {  
  client: 'pg',  
  connection: process.env.DATABASE_URL || {  
    user: 'me',  
    database: 'my_app',  
  },  
};
```

js

You can also use an async function to get connection details for your configuration. This is useful when you need to fetch credentials from a secure location like vault.

```
const getPassword = async () => {
  // TODO: implement me
  return 'my_pass';
};

module.exports = {
  client: 'pg',
  connection: async () => {
    const password = await getPassword();
    return { user: 'me', password };
  },
  migrations: {},
};
```

js

Environment configuration

```
module.exports = {
  development: {
    client: 'pg',
    connection: { user: 'me', database: 'my_app' },
  },
  production: {
    client: 'pg',
    connection: process.env.DATABASE_URL,
  },
};
```

js

Custom migration

You may provide a custom migration stub to be used in place of the default option.

```
module.exports = {
  client: 'pg',
  migrations: {
    stub: 'migration.stub',
  },
};
```

js

```
},  
};
```

Custom migration name

You may provide a custom migration name to be used in place of the default option.

```
module.exports = {  
  client: 'pg',  
  migrations: {  
    getNewMigrationName: (name) => {  
      return `${+new Date()}-${name}.js`;  
    },  
  },  
};
```

js

Generated migration extension

You can control extension of generated migrations.

```
module.exports = {  
  client: 'pg',  
  migrations: {  
    extension: 'ts',  
  },  
};
```

js

Knexfile in other languages

Knex uses [Liftoff](#) to support knexfile written in other compile-to-js languages.

Depending on the language, this may require you to install additional dependencies. The complete list of dependencies for each supported language can be found [here](#).

Most common cases are typescript (for which [typescript](#) and [ts-node](#) packages are recommended), and coffeescript (for which [coffeescript](#) dependency is required).

If you don't specify the extension explicitly, the extension of generated migrations/seed files will be inferred from the knexfile extension

Migration API

`knex.migrate` is the class utilized by the knex migrations cli.

Each method takes an optional `config` object, which may specify the following properties:

- `directory`: a relative path to the directory containing the migration files. Can be an array of paths (default `./migrations`)
- `extension`: the file extension used for the generated migration files (default `js`)
- `tableName`: the table name used for storing the migration state (default `knex_migrations`)
- `schemaName`: the schema name used for storing the table with migration state (optional parameter, only works on DBs that support multiple schemas in a single DB, such as PostgreSQL)
- `disableTransactions`: don't run migrations inside transactions (default `false`)
- `disableMigrationsListValidation`: do not validate that all the already executed migrations are still present in migration directories (default `false`)
- `sortDirsSeparately`: if true and multiple directories are specified, all migrations from a single directory will be executed before executing migrations in the next folder (default `false`)
- `loadExtensions`: array of file extensions which knex will treat as migrations. For example, if you have typescript transpiled into javascript in the same folder, you want to execute only javascript migrations. In this case, set `loadExtensions` to `['.js']` (Notice the dot!) (default `['.co', '.coffee', '.eg', '.iced', '.js', '.litcoffee', '.ls', '.ts']`)
- `migrationSource`: specify a custom migration source, see [Custom Migration Source](#) for more info (default filesystem)

Transactions in migrations

By default, each migration is run inside a transaction. Whenever needed, one can disable transactions for all migrations via the common migration config option `config.disableTransactions` or per-migration, via exposing a boolean property `config.transaction` from a migration file:

js

```

exports.up = function (knex) {
  return knex.schema
    .createTable('users', function (table) {
      table.increments('id');
      table.string('first_name', 255).notNullable();
      table.string('last_name', 255).notNullable();
    })
    .createTable('products', function (table) {
      table.increments('id');
      table.decimal('price').notNullable();
      table.string('name', 1000).notNullable();
    });
};

exports.down = function (knex) {
  return knex.schema.dropTable('products').dropTable('users');
};

exports.config = { transaction: false };

```

The same config property can be used for enabling transaction per-migration in case the common configuration has `disableTransactions: false`.

Sqlite3-specific concerns

Disabling / enabling foreign key checking has no effect within a transaction on sqlite3, so features such as `.dropColumn()` cannot be emulated correctly inside of a migration-level transaction (and can even cause data loss). Knex now throws an error in these cases.

It is recommended to write migrations intended for sqlite using `transaction: false` in your migration file, and use explicit transactions where possible if you want them. You may supply `enforceForeignCheck: false` in the call to `knex.transaction()`, which will disable foreign key checking before creating the transaction. If foreign key checking was enabled before the transaction, `PRAGMA foreign_key_check` will be run before committing the transaction to ensure the data is sound:

js

```

exports.up = function (knex) {
  return knex.transaction(function (trx) {
    return knex.schema.alterTable('users', function (table) {
      table.string('website');
    });
  });
};

```

```

    });
  });
};

exports.down = function (knex) {
  return knex.transaction(
    function (trx) {
      return knex.schema.alterTable('users', function (table) {
        table.dropColumn('website');
      });
    },
    { enforceForeignCheck: false }
  );
};
exports.config = { transaction: false };

```

make

```
knex.migrate.make(name, [config])
```

Creates a new migration, with the name of the migration being added.

latest

```
knex.migrate.latest([config])
```

Runs all migrations that have not yet been run.

If you need to run something only after all migrations have finished their execution, you can do something like this:

```

knex.migrate
  .latest()
  .then(function () {
    return knex.seed.run();
  })
  .then(function () {
    // migrations are finished
  });

```

js

rollback

```
knex.migrate.rollback([config], [all])
```

Rolls back the latest migration group. If the `all` parameter is truthy, all applied migrations will be rolled back instead of just the last batch. The default value for this parameter is `false`.

up

```
knex.migrate.up([config])
```

Runs the specified (by `config.name` parameter) or the next chronological migration that has not yet be run.

down

```
knex.migrate.down([config])
```

Will undo the specified (by `config.name` parameter) or the last migration that was run.

currentVersion

```
knex.migrate.currentVersion([config])
```

Retrieves and returns the current migration version, as a promise. If there aren't any migrations run yet, returns "none" as the value for the currentVersion.

list

```
knex.migrate.list([config])
```

Will return list of completed and pending migrations

unlock

```
knex.migrate.forceFreeMigrationsLock([config])
```

Forcibly unlocks the migrations lock table, and ensures that there is only one row in it.

Notes about locks

A lock system is there to prevent multiple processes from running the same migration batch in the same time. When a batch of migrations is about to be run, the migration system first tries to get a lock using a `SELECT ... FOR UPDATE` statement (preventing race conditions from happening). If it can get a lock, the migration batch will run. If it can't, it will wait until the lock is released.

Please note that if your process unfortunately crashes, the lock will have to be *manually* removed with `knex migrate:unlock` in order to let migrations run again.

The locks are saved in a table called "`tableName_lock`"; it has a column called `is_locked` that `knex migrate:unlock` sets to `0` in order to release the lock. The `index` column in the lock table exists for compatibility with some database clusters that require a primary key, but is otherwise unused. There must be only one row in this table, or an error will be thrown when running migrations: "Migration table is already locked". Run `knex migrate:unlock` to ensure that there is only one row in the table.

Custom migration sources

Knex supports custom migration sources, allowing you full control of where your migrations come from. This can be useful for custom folder structures, when bundling with webpack/browserify and other scenarios.

```
// Create a custom migration source class
class MyMigrationSource {
  // Must return a Promise containing a list of migrations.
  // Migrations can be whatever you want,
  // they will be passed as arguments to getMigrationName
  // and getMigration
  getMigrations() {
    // In this example we are just returning migration names
    return Promise.resolve(['migration1']);
  }

  getMigrationName(migration) {
    return migration;
  }
}
```

js

```

getMigration(migration) {
  switch (migration) {
    case 'migration1':
      return {
        up(knex) {
          /* ... */
        },
        down(knex) {
          /* ... */
        },
      };
    }
  }
}

// pass an instance of your migration source as knex config
knex.migrate.latest({
  migrationSource: new MyMigrationSource(),
});

```

Webpack migration source example

An example of how to create a migration source where migrations are included in a webpack bundle.

```

const path = require('path');

class WebpackMigrationSource {
  constructor(migrationContext) {
    this.migrationContext = migrationContext;
  }

  getMigrations() {
    return Promise.resolve(this.migrationContext.keys().sort());
  }

  getMigrationName(migration) {
    return path.parse(migration).base;
  }

  getMigration(migration) {
    return this.migrationContext(migration);
  }
}

```

js

```

    }
  }

  // pass an instance of your migration source as knex config
  knex.migrate.latest({
    migrationSource: new WebpackMigrationSource(
      require.context('./migrations', false, /\.js$/)
    ),
  });

  // with webpack >=5, require.context will add
  // both the relative and absolute paths to the context
  // to avoid duplicate migration errors, you'll need
  // to filter out one or the other this example filters
  // out absolute paths, leaving only the relative
  // ones(./migrations/*.js):
  knex.migrate.latest({
    migrationSource: new WebpackMigrationSource(
      require.context('./migrations', false, /^\.\/.*\.js$/)
    ),
  });

```

ECMAScript modules (ESM) Interoperability

ECMAScript Module support for knex CLI's configuration, migration and seeds enabled by the `--esm` flag, ECMAScript Interoperability is provided by the ['esm'](#) module.

You can find [here](#) more information about 'esm' superpowers.

Node 'mjs' files are handled by NodeJS own import mechanics and do not require the use of the '--esm' flag.

But you might need it anyway for Node v10 under certain scenarios.

You can find details about NodeJS ECMAScript modules [here](#)

While it is possible to mix and match different module formats (extensions) between your knexfile, seeds and migrations, some format combinations will require specific NodeJS versions, *Notably mjs/cjs files will follow NodeJS import and require restrictions.*

You can see [here](#) many possible scenarios,
and [here](#) some sample configurations

Node v10.* require the use of the '--experimental-module' flag in order to use the 'mjs' or 'cjs' extension.

```
# launching knex on Node v10 to use mjs/cjs modules
node --experimental-modules ./node_modules/.bin/knex $@
```

bash

When using migration and seed files with '.cjs' or '.mjs' extensions, you will need to specify that explicitly:

```
/**
 * knexfile.mjs
 */
export default {
  migrations: {
    // ... client, connection,etc ....
    directory: './migrations',
    loadExtensions: ['.mjs'], //
  },
};
```

ts

When using '.mjs' extensions for your knexfile and '.js' for the seeds/migrations, you will need to specify that explicitly.

```
/**
 * knexfile.mjs
 */
export default {
  migrations: {
    // ... client, connection,etc ....
    directory: './migrations',
    loadExtensions: ['.js'], // knex will search for 'mjs' file by default
  },
};
```

ts

For the knexfile you can use a default export,
it will take precedence over named export.

```

/**
 * filename: knexfile.js
 * For the knexfile you can use a default export
 */
export default {
  client: 'sqlite3',
  connection: {
    filename: '../test.sqlite3',
  },
  migrations: {
    directory: './migrations',
  },
  seeds: {
    directory: './seeds',
  },
};

/**
 * filename: knexfile.js
 * Let knex find the configuration by providing named exports,
 * but if exported a default, it will take precedence, and it will be used inst
 */
const config = {
  client: 'sqlite3',
  connection: {
    filename: '../test.sqlite3',
  },
  migrations: {
    directory: './migrations',
  },
  seeds: {
    directory: './seeds',
  },
};
/** this will be used, it has precedence over named export */
export default config;
/** Named exports, will be used if you didn't provide a default export */
export const { client, connection, migrations, seeds } = config;

```

Seed and migration files need to follow Knex conventions

```

// file: seed.js
/**

```



```

* Same as with the CommonJS modules
* You will need to export a "seed" named function.
* */
export function seed(knex) {
  // ... seed logic here
}

// file: migration.js
/**
* Same as the CommonJS version, the migration file should export
* "up" and "down" named functions
*/
export function up(knex) {
  // ... migration logic here
}
export function down(knex) {
  // ... migration logic here
}

```

Seed API

`knex.seed` is the class utilized by the knex seed CLI.

Each method takes an optional `config` object, which may specify the following properties:

- `directory`: a relative path to the directory containing the seed files. Can be an array of paths (default `./seeds`)
- `loadExtensions`: array of file extensions which knex will treat as seeds. For example, if you have typescript transpiled into javascript in the same folder, you want to execute only javascript seeds. In this case, set `loadExtensions` to `['.js']` (Notice the dot!) (default `['.co', '.coffee', '.eg', '.iced', '.js', '.litcoffee', '.ls', '.ts']`)
- `recursive`: if true, will find seed files recursively in the directory / directories specified
- `specific`: a specific seed file or an array of seed files to run from the seeds directory, if its value is `undefined` it will run all the seeds (default `undefined`). If an array is specified, seed files will be run in the same order as the array

- `sortDirsSeparately` : if true and multiple directories are specified, all seeds from a single directory will be executed before executing seeds in the next folder (default `false`)
- `seedSource` : specify a custom seed source, see [Custom Seed Source](#) for more info (default filesystem)
- `extension` : extension to be used for newly generated seeds (default `js`)
- `timestampFilenamePrefix` : whether timestamp should be added as a prefix for newly generated seeds (default `false`)

make

```
knex.seed.make(name, [config])
```

Creates a new seed file, with the name of the seed file being added. If the seed directory config is an array of paths, the seed file will be generated in the latest specified.

run

```
knex.seed.run([config])
```

Runs all seed files for the current environment.

Custom seed sources

Knex supports custom seed sources, allowing you full control of where your seeds come from. This can be useful for custom folder structures, when bundling with webpack/browserify and other scenarios.

```
// Create a custom seed source class
class MySeedSource {
  // Must return a Promise containing a list of seeds.
  // Seeds can be whatever you want, they will be passed as
  // arguments to getSeed
  getSeeds() {
    // In this example we are just returning seed names
    return Promise.resolve(['seed1']);
  }
}
```

js

```
getSeed(seed) {  
  switch (seed) {  
    case 'seed1':  
      return (knex) => {  
        /* ... */  
      };  
    }  
  }  
}  
  
// pass an instance of your seed source as knex config  
knex.seed.run({ seedSource: new MySeedSource() });
```

[✎ Edit this page on GitHub](#)

Last Updated: 6/26/26, 10:59 PM

Previous page
[Interfaces](#)

Next page
[Extending](#)